Licence Informatique/Miage S6 – XML - TP n°1

Lors de ce TP et des suivants, nous serons amenés à utiliser *xmllint* qui est un petit utilitaire en ligne de commande pour valider et manipuler des fichiers xml. Pour connaître le manuel d'utilisation vous pouvez taper la commande *man xmllint*.

Exercice 1 :

Observez le fichier exo1.xml suivant :

```
<? xml version="1.0"?>
<!-- this is a note -->
<note date=3 janvier>
<to>Bob</To>
<from>Alice</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
</note>
<note date="5 janvier" <!-- this is another note --> >
<to>Alice</to>
<from>Bob
<body>No problem & see you soon</body>
</note>
<note />
```

1. Ce document est-il bien formé (i.e. respecte-t-il la syntaxe XML) ?
2. S'il ne l'est pas, corrigez les erreurs.
3. Vérifier que le document obtenu est correct en tapant la commande *xmllint exo1.xml*.

Exercice 2 :

Corrigez le fichier exo2.xml suivant. Vérifier que le document corrigé est valide en tapant la commande *xmllint --noout exo2.xml && echo "tout est OK"*.

```
<?xml version="1.0" encoding='UTF-8' standalone="yes"?>
<!-- tableau périodique des éléments --!>
<classification_atomique>
<atome ligne="3" colonne="2">
<nom>magnésium</nom>
<symbole>Mg</symbole>
<numero>12</numero>
<masse>24</masse>
<type>métal alcalino-terreux</type>
</atome>
<atome ligne="3" /colonne="3">
<nom>aluminium</nom>
<symbole>Al</symbole>
<numero>13</numero>
<masse>27<masse/>
```

```
<type>autre métal</type>
</atome>
<atome ligne="3" colonne="4">
<nom>silicium</nom>
<symbole>Si</symbole>
</numero>14<numero>
<masse>28</masse>
<type>non métal</type>
<atome ligne=3 colonne="8">
<nom>argon</nom>
<symbole>Ar</symbole>
</atome>
<numero>18</numero>
<masse>40</masse>
<type>gaz rare</type>
</atome>
</classification-atomique>
<classification_atomique>
<atome ligne="1" colonne="1">
<nom>hydrogène</nom>
<symbole>H</symbole>
<numero>1</numero>
<masse>1</masse>
<type/>
</classification-atomique>
```

Exercice 3 :

On souhaite écrire un livre en utilisant le formalisme XML. Le livre est structuré en sections, en chapitres (au moins 2 par section) et en paragraphes (au moins 2 par chapitre). Le livre doit contenir la liste des auteurs (avec nom et prénom).
Le livre ainsi que chaque section et chaque chapitre doivent posséder un titre. Chaque paragraphe doit contenir du texte.

Proposez un exemple de livre formaté comme un document XML respectant les contraintes susmentionnées avec 3 sections. Vérifier la validité de votre document via la commande *xmllint*.

Exercice 4 :

Créez un document exo4.xml listant votre collection de Cds à l'aide de la DTD exo4.dtd ci-dessous.

```
<!ELEMENT cdtheque (cd*) >
<!ATTLIST cdtheque date CDATA #REQUIRED>
<!ATTLIST cdtheque proprietaire CDATA #REQUIRED>
<!ELEMENT cd (titre,artiste,style?)>
<!ELEMENT titre (#PCDATA)>
<!ELEMENT style (#PCDATA)>
<!ELEMENT artiste (#PCDATA)>
```

Pour valider un document xml, avec un doctype, si le document xml ne contient aucune référence à un doctype :

xmllint --noout --dtdvalid exo4.dtd exo4.xml && echo "respecte la dtd"

Dans le cas où le document contient une référence à un doctype taper simplement :

xmllint exo4.xml --valid --noout && echo "respecte la dtd"

Exercice 5 :

Vérifiez la validité des 5 fichiers XML ,donnés plus bas, relativement à la DTD pingpong.dtd ci-dessous . Si un fichier est invalide, corrigez-le pour le rendre valide, en minimisant le nombre de corrections.

```
<!ELEMENT racine (debut? , suite+)>
<!ELEMENT debut (ping? , pong*)>
<!ELEMENT suite (ping , pong)>
<!ELEMENT ping (#PCDATA)>
<!ELEMENT pong (#PCDATA)>
```

pp1.xml :

```
<racine>
   <debut>
      <ping/><pong/>
   </debut>
   <suite>
      <ping/><pong/>
   </suite>
   <suite>
      <ping/><pong/>
   </suite>
</racine>
```

pp2.xml :

```
<racine>
   <debut/>
   <suite>
      <ping/><pong/>
   </suite>
</racine>
```

pp3.xml :

```
<racine>
   <debut>
      <ping/><ping/>
      <pong/><pong/>
   </debut>
   <suite>
      <ping/><pong/>
```

```
   </suite>
</racine>
```

pp4.xml :

```
<racine>
   <suite>
      <pong/><pong/>
   </suite>
   <suite>
      <ping/><pong/>
   </suite>
</racine>
```

pp5.xml :

```
<racine>
   <debut/>
   <suite/>
</racine>
```

Exercice 6 :

Proposez un fichier xml exo6.xml contenant au moins 6 éléments qui soit valide pour le doctype exo6.dtd suivant :

```
<!ELEMENT toto (titi | (toto,toto,titi)) >
<!ELEMENT titi (#PCDATA) >
```

Exercice 7 :

Proposez une DTD dialogue.dtd pour le fichier xml ci-dessous et vérifiez la validité du document xml au regard ce cette DTD en utilisant la commande xmllint. Au moins deux attributs ne devront pas être de type CDATA.

```
<?xml version='1.0' encoding='UTF-8' standalone='no' ?>

<!DOCTYPE dialogue SYSTEM "dialogue.dtd">

<!-- extrait de Tartuffe de Molière -->
<dialogue>
<situation>acte I, Scene 1 : madame pernelle et flipote sa
servante,elmire, mariane, dorine, damis, cléante.</situation>
<replique>
<personnage attitude='pressée' geste='marchant vite' >madame
pernelle</personnage>
<texte ton='fort' >Allons, Flipote, allons, que d'eux je me
délivre.</texte>
</replique>
```

```
<replique>
<personnage attitude='essouflée' >elmire</personnage>
<texte>Vous marchez d'un tel pas qu'on a peine à vous
suivre.</texte>
</replique>
<replique>
<personnage attitude='agacée' >madame pernelle</personnage>
<texte>Laissez, ma bru, laissez, ne venez pas plus loin: Ce sont
toutes façons dont je n'ai pas besoin.</texte>
</replique>
<replique>
<personnage attitude='étonnée' >elmire</personnage>
<texte>De ce que l'on vous doit envers vous on s'acquitte, Mais
ma mère, d'où vient que vous sortez si vite;</texte>
</replique>
</dialogue>
```

Exercice 8 :

Proposez une DTD pour le fichier xml ci-dessous et vérifiez la validité du document xml au regard ce cette DTD en utilisant la commande xmllint.

```
<?xml version="1.0" encoding="UTF-8"?>
<étudiant>
<prénom>Jean</prénom>
<nom>Duschmol</nom>
<num_etudiant>21867809</num_etudiant>
<parcours>
<année year="3">
<module>
<titre> Programmation orientée objet </titre>
<abbréviation>POO</abbréviation>
<résultat>
<note>5</note>
<date>13/06/2017</date>
</résultat>
</module>
<module>
<titre>XML</titre>
<résultat>
<note>10</note>
<date>13/06/2017</date>
</résultat>
<résultat>
<note>9</note>
<date>12/03/2017</date>
<observation> Triche durant l'examen </observation>
</résultat>
</module>
</année>
</parcours>
</étudiant>
```

Exercice 9 :

Proposez une DTD pour le fichier xml ci-dessous et vérifiez la validité du document xml au regard ce cette DTD en utilisant la commande xmllint.

```
<?xml version="1.0" encoding="UTF-8"?>
<email>
<from> yoann.dieudonne@u-picardie.fr </from>
<to> jean.duschmol@u-picardie.fr </to>
<contenu>
Cher <person> Jean Dushmol </person>,
Veuillez bien vouloir trouver ci-dessous une série d'exercice
concernant le module <module> XML </module>:
<exercises>
<exercise>
<sujet> DTD </sujet>
<description> Ecrire une DTD permettant de valider blablaba
</description>
</exercise>
<exercise>
<sujet> XPath </sujet>
<description> via l'option xpath de la commande xmmlint, trouvez
tous les éléments ayant un attribut age supérieur à 18
</description>
</exercise>
</exercises>
Cordialement,
<person> Yoann Dieudonné </person>
</contenu>
</email>
```

Exercice 10 :

Ecrire un document xml vérifiant le doctype suivant :

```
<!ELEMENT BIBLIOTHEQUE (LIVRE,LIVRE+,AUTEUR, AUTEUR+)>
 <!ELEMENT LIVRE (TITRE, OBSERVATION?)>
 <!ATTLIST LIVRE isbn ID #REQUIRED
                 prix CDATA #IMPLIED
                 auteurs IDREFS #REQUIRED>
 <!ENTITY % sexe "sexe (h|f) #REQUIRED">
 <!ELEMENT TITRE (#PCDATA)>
 <!ELEMENT OBSERVATION (#PCDATA)>
 <!ELEMENT AUTEUR (PRENOM, NOM)>
 <!ATTLIST AUTEUR Identite ID #REQUIRED>
 <!ATTLIST AUTEUR livres IDREFS #REQUIRED %sexe;>
 <!ELEMENT PRENOM (#PCDATA)>
 <!ELEMENT NOM (#PCDATA)>
```

Exercice 11 :

Considérez les deux doctypes suivants, puis répondez aux 4 questions qui suivent. Pour chacune d'entre elles utilisez la commande xmllint pour vérification.

<!-- DOCTYPE 1 --> <!ELEMENT titi (titi|(toto|tutu)*)> <!ELEMENT toto (tutu,toto?)+> <!ELEMENT tutu (#PCDATA|tutu)*>	<!-- DOCTYPE 2 --> <!ELEMENT titi (toto?,tutu)> <!ELEMENT toto (tutu|toto)+> <!ELEMENT tutu (#PCDATA)>

1. Ecrire un fichier xml satisfaisant le premier doctype mais pas le second.

2. Ecrire un fichier xml satisfaisant le second doctype mais pas le premier.

3. Ecrire un fichier xml satisfaisant les deux doctypes.

4. Ecrire un fichier xml satisfaisant les deux doctypes et qui contient au moins 4 éléments tutu.

Exercice 12 :

Un aspect qui n'a pas été abordé jusque maintenant est le fait que toute DTD doit être déterministe. En d'autres termes, à chaque fois que le parseur tombe sur une balise dans le document xml, il ne doit pas aller analyser la suite du document xml pour savoir quelle règle est activée. Cela permet un gain d'efficacité considérable pour vérifier la validité d'un document xml au regard d'une DTD (on obtient une complexité linéaire en nombre d'éléments au lieu d'une complexité cubique).

Exemple de règle non-déterministe :

```
<!ELEMENT A ((B,C)|(B,D)) ?>
```

Quand le parseur tombe la première fois sur une balise B fille d'une balise A, il ne sait pas si la règle activée est le B de (B,C) ou le B de (B,D) : il faudrait voir la suite du document, ce qui rend la DTD non déterministe.
Un autre exemple :

```
<!ELEMENT X (Y*,Y) ?>
```

Quand le parseur tombe la première fois sur une balise Y fille d'une balise X, il ne sait pas si la règle activée est celle du Y* ou celle du Y : il faudrait voir la suite du document, ce qui rend la DTD non déterministe.

Pour les deux exemple non déterministes du dessus, on a des règles déterministes équivalentes. Pour le premier, on a

```
<!ELEMENT A (B,(C|D)) ?>
```

Et pour le second, on a :

```
<!ELEMENT X (Y,Y*) ?>
```

Il faut savoir qu'il n'est pas toujours possible de traduire une règle non déterministe en une règle déterministe équivalente (cf. cours de langage formel). Il faut alors dans ce cas généraliser ou contraindre la règle initiale…

Définir une DTD bidule.dtd permettant de valider les documents xml ayant pour balise racine un élément TRUC qui respecte les règles suivantes :

- l'élement TRUC ne peut avoir que des fils A, B, C.
- Les fils doivent apparaître dans l'ordre alphabétique : par exemple A ne peut pas apparaître après C.
- l'élement TRUC ne peut pas avoir de double. Par exemple, s'il y a un sous-élément A, TRUC ne peut pas avoir de second sous-élément A.
- l'élément TRUC n'est pas obligé d'avoir 3 fils, il doit seulement en avoir au moins un.

Vérifiez la validité de votre DTD avec des documents XML respectant ces contraintes en utilisant la commande xmllint. Assurez-vous de ne pas voir le message suivant apparaître « validity error : Content model of section is not determinist:... », auquel cas il faut revoir la DTD pour la rendre déterministe.

Exercice 13 :

Voici une série de 10 documents XML.

```
<X>
  <Y algo="A307"/>
  <Y algo="a380"/>
  <Z/>
</X>
```

```
<X>
  <Y algo="307"/>
  <Z/>
</X>
```

```
<X>
  <Y algo="A307"/>
  <Z/>
  <Z/>
</X>
```

```
<X>
  <Y algo="A307"/>
  <Z/>
  <Y algo="C307"/>
  <Z/>
  <Y/>
</X>
```

```
<X machin="truc">
  <Y algo="A307">
    blablabla
  </Y>
  <Z/>
  <Y algo="a307"/>
  <Z>
    <Z/>
  </Z>
</X>
```

```
<X machin="truc">
  <Y algo="a307">
    blablabla
  </Y>
  <Z/>
  <Y algo="b307"/>
  <Z>
    <Z/>
    <Z/>
    <Z/>
    <Z/>
  </Z>
</X>
```

```
<X machin="truc">
  <Y algo="a307">
    blablabla
  </Y>
  <Z/>
  <Y algo="b307"/>
  <Z>
    <Z>blablabla</Z>
    <Z/>
  </Z>
</X>
```

```
<X machin="12">
  <Y algo="A307">
    blablabla
  </Y>
  <Z/>
  <Y algo="a380"/>
  <Z>
    <Z/>
    <Z/>
  </Z>
</X>
```

```
<X machin="truc">
  <Y algo="A307">
    blablabla
  </Y>
  <Z/>
  <Y algo="A307"/>
  <Z>
    <Z/>
    <Z/>
  </Z>
</X>
```

```
<X machin="truc">
  <Y algo="a307">
    <Z/>
  </Y>
  <Z/>
  <Y algo="b307"/>
  <Z>
    <Z/>
    <Z/>
  </Z>
</X>
```

Ecrire une DTD qui invalide les 10 documents du dessus mais qui valide les documents ci-dessous. Vérifiez via xmllint.

```
<X machin="truc">
  <Y algo="A307">
    blablabla
  </Y>
  <Z/>
  <Y algo="b307"/>
  <Z>
    <Z/>
    <Z/>
  </Z>
</X>
```

```
<X machin="chose36">
  <Y algo="badaboum"/>
  <Z/>
</X>
```

```
<X>
  <Y algo="X"/>
  <Z/>
</X>
```